

4-解码器知识蒸馏

笔记本: LLM_practice

创建时间: 2025/2/5 9:22

更新时间: 2025/4/28 20:10

作者: alexritchie@163.com

URL: https://github.com/wyf3/llm_related/tree/main/knowledge_distillation_llm

4-解码器知识蒸馏

对于Transformer Encoder进行蒸馏，将待压缩的模型作为教师模型，将体积更小的模型作为学生模型，让学生模型在教师模型的监督下进行优化，将学生模型学习到教师模型的概率分布，通过kl散度进行控制。

大模型多采用Transformer Decoder结构，对于大模型的知识蒸馏，主要分为两种：

一、黑盒知识蒸馏

使用大模型生成数据，通过这些数据去微调更小的模型，来达到蒸馏的目的。缺点是蒸馏效率低，优点是实现简单。

Step 1: 写好Prompt，提交给大模型，大模型作为教师模型产生Response。

Step 2: 小模型作为Student，以大模型的输出作SFT。

二、白盒知识蒸馏

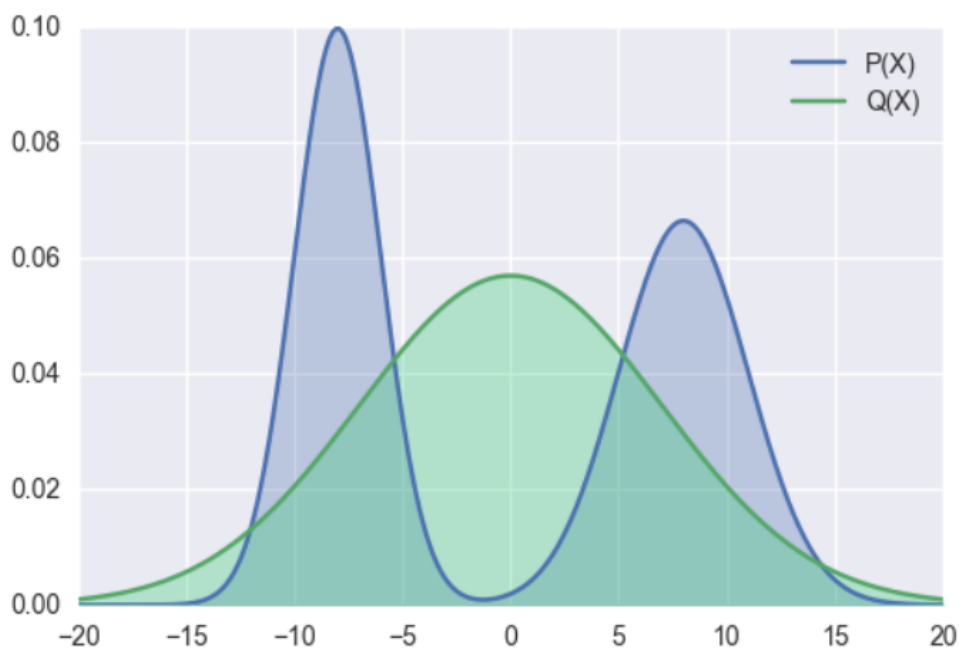
获取学生模型和教师模型的输出概率分布（或者中间隐藏层的概率分布），通过kl散度将学生模型的概率分布向教师模型对齐。下面主要介绍和测试白盒知识蒸馏：白盒知识蒸馏主要在于模型分布的对齐，模型分布对齐主要依赖kl散度，对于kl散度的使用又有如下几种方式：

2.1 前向KL散度

也就是我们通常意义上的KL散度

$$\hat{q} = \arg \min_q \int p(x) \log \frac{p(x)}{q(x)} dx$$

p 为教师模型的概率分布， q 为学生模型的概率分布，minillm论文中提到前向kl散度可能会使学生模型高估教师模型中概率比较低的位置，结合公式来看，当 p 增大时，为了使得KL散度小，则 q 也需要增大，但是当 p 趋于0时，无论 q 取任何值，KL散度都比较小，因为此时 $p(x)\log((p(x)/q(x)))$ 的大小主要受 $p(x)$ 控制，这样起不到优化 q 分布的效果，可能会使 q 分布高估 p 分布中概率低的位置。下图展示了前向kl散度的拟合情况，前向kl散度是一种均值搜索，更倾向于拟合多峰。



代码如下：

```
# 前向kl散度
def compute_fkl(
    logits,
    teacher_logits,
    target,
    padding_id,
    reduction="sum",
    temp=1.0,
):
    logits = logits / temp
    teacher_logits = teacher_logits / temp

    log_probs = torch.log_softmax(logits, -1, dtype=torch.float32)
    teacher_probs = torch.softmax(teacher_logits, -1, dtype=torch.float32)
    teacher_log_probs = torch.log_softmax(teacher_logits, -1,
    dtype=torch.float32)
    kl = (teacher_probs * (teacher_log_probs - log_probs))
```

```

kl = kl.sum(-1)
if reduction == "sum":
    pad_mask = target.eq(padding_id)
    kl = kl.masked_fill_(pad_mask, 0.0)
    kl = kl.sum()

return kl

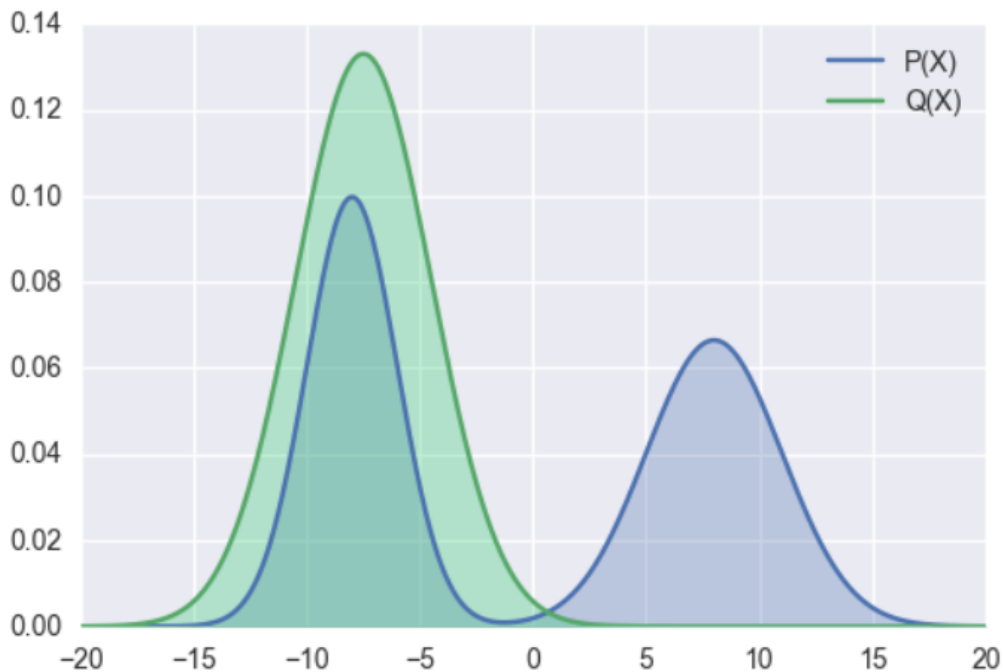
```

2.2 反向KL散度

为了缓解前向kl散度的缺点，一些工作使用了反向kl散度。

$$\hat{q} = \arg \min_q \int q(x) \log \frac{q(x)}{p(x)} dx$$

p为教师模型的概率分布，q为学生模型的概率分布，当p趋于零时，为了使KL散度小，q也需趋于0。minillm论文中说对于大模型的知识蒸馏，反向KL散度优于前向KL散度，但是也有其他论文说反向KL散度不一定比前向KL散度更优，实际选择中，可能要基于实验驱动。反向KL散度是一种选择。



代码如下：

```

# 计算反向kl散度
def compute_rkl(
    logits,
    teacher_logits,
    target,

```

```

        padding_id,
        reduction="sum",
        temp=1.0
    ):
        logits = logits / temp
        teacher_logits = teacher_logits / temp

        probs = torch.softmax(logits, -1, dtype=torch.float32)
        log_probs = torch.log_softmax(logits, -1, dtype=torch.float32)
        teacher_log_probs = torch.log_softmax(teacher_logits, -1,
dtype=torch.float32)
        kl = (probs * (log_probs - teacher_log_probs))
        kl = kl.sum(-1)
        if reduction == "sum":
            pad_mask = target.eq(padding_id)
            kl = kl.masked_fill_(pad_mask, 0.0)
            kl = kl.sum()
        return kl

```

2.3 偏向前kl散度

```

# 计算偏向前kl散度
def compute_skewed_fkl(
    logits,
    teacher_logits,
    target,
    padding_id,
    reduction="sum",
    temp=1.0,
    skew_lambda=0.1
):
    logits = logits / temp
    teacher_logits = teacher_logits / temp

    probs = torch.softmax(logits, -1, dtype=torch.float32)
    teacher_probs = torch.softmax(teacher_logits, -1, dtype=torch.float32)
    mixed_probs = skew_lambda * teacher_probs + (1 - skew_lambda) * probs
    mixed_log_probs = torch.log(mixed_probs)
    teacher_log_probs = torch.log_softmax(teacher_logits, -1,
dtype=torch.float32)
    kl = (teacher_probs * (teacher_log_probs - mixed_log_probs))
    kl = kl.sum(-1)

```

```

if reduction == "sum":
    pad_mask = target.eq(padding_id)
    kl = kl.masked_fill_(pad_mask, 0.0)
    kl = kl.sum()

return kl

```

2.4 偏向反kl散度

```

# 计算偏向反kl散度
def compute_skewed_rkl(
    logits,
    teacher_logits,
    target,
    padding_id,
    reduction="sum",
    temp=1.0,
    skew_lambda=0.1
):
    logits = logits / temp
    teacher_logits = teacher_logits / temp

    probs = torch.softmax(logits, -1, dtype=torch.float32)
    teacher_probs = torch.softmax(teacher_logits, -1, dtype=torch.float32)
    mixed_probs = (1 - skew_lambda) * teacher_probs + skew_lambda * probs
    mixed_log_probs = torch.log(mixed_probs)
    log_probs = torch.log_softmax(logits, -1, dtype=torch.float32)
    kl = (probs * (log_probs - mixed_log_probs))
    kl = kl.sum(-1)

    if reduction == "sum":
        pad_mask = target.eq(padding_id)
        kl = kl.masked_fill_(pad_mask, 0.0)
        kl = kl.sum()

    return kl

```

六、代码执行

这个例子中，我们会使用Qwen2.5-7B版本作为教师模型，随后在alpaca数据集中产生软标签，用于指导学生模型的蒸馏。

6.1 载入数据集

读取alpaca数据集，请注意，在教师微调和知识蒸馏过程中都是需要的。

```
# process dataset
alpaca_prompt = """Below is an instruction that describes a task, paired with
an input that provides further context. Write a response that appropriately
completes the request.

### Instruction:
{}

### Input:
{}

### Response:
{} """

EOS_TOKEN = tokenizer.eos_token # Must add EOS_TOKEN
def formatting_prompts_func(examples):
    instructions = examples["instruction"]
    inputs        = examples["input"]
    outputs       = examples["output"]
    texts = []
    for instruction, input, output in zip(instructions, inputs, outputs):
        # Must add EOS_TOKEN, otherwise your generation will go on forever!
        text = alpaca_prompt.format(instruction, input, output) + EOS_TOKEN
        texts.append(text)
    return { "text" : texts, }

from datasets import load_dataset
dataset = load_dataset("yahma/alpaca-cleaned", split = "train")
dataset = dataset.map(formatting_prompts_func, batched = True,)
```

6.2 教师模型的微调

读取Qwen2.5-7B模型作为教师模型

```
from unsloth import FastLanguageModel
import torch
```

```

max_seq_length = 2048 # Choose any! We auto support RoPE Scaling internally!
dtype = None # None for auto detection. Float16 for Tesla T4, V100, Bfloat16
                for Ampere+
load_in_4bit = True # Use 4bit quantization to reduce memory usage. Can be
                False.

model, tokenizer = FastLanguageModel.from_pretrained(
    # Can select any from the below:
    # "unsloth/Qwen2.5-0.5B", "unsloth/Qwen2.5-1.5B", "unsloth/Qwen2.5-3B"
    # "unsloth/Qwen2.5-14B", "unsloth/Qwen2.5-32B", "unsloth/Qwen2.5-72B",
    # And also all Instruct versions and Math. Coding versions!
    model_name = "unsloth/Qwen2.5-7B",
    max_seq_length = max_seq_length,
    dtype = dtype,
    load_in_4bit = load_in_4bit,
    # token = "hf...", # use one if using gated models like meta-
    # llama/Llama-2-7b-hf
)

model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # Choose any number > 0 ! Suggested 8, 16, 32, 64, 128
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj",],
    lora_alpha = 16,
    lora_dropout = 0, # Supports any, but = 0 is optimized
    bias = "none",    # Supports any, but = "none" is optimized
    # [NEW] "unsloth" uses 30% less VRAM, fits 2x larger batch sizes!
    use_gradient_checkpointing = "unsloth", # True or "unsloth" for very long
    # context
    random_state = 3407,
    use_rslora = False, # We support rank stabilized LoRA
    loftq_config = None, # And LoftQ
)

```

6.3 训练保存模型

```

from trl import SFTTrainer
from transformers import TrainingArguments
from unsloth import is_bfloat16_supported

```

```

training_args = TrainingArguments(
    per_device_train_batch_size = 2,
    gradient_accumulation_steps = 4,
    warmup_steps = 5,
    # num_train_epochs = 1, # Set this for 1 full training run.
    max_steps = 600,
    learning_rate = 2e-4,
    fp16 = not is_bfloat16_supported(),
    bfloat16 = is_bfloat16_supported(),
    logging_steps = 1,
    optim = "adamw_8bit",
    weight_decay = 0.01,
    lr_scheduler_type = "linear",
    seed = 3407,
    output_dir = "outputs",
    report_to = "none", # Use this for WandB etc
)

trainer = SFTTrainer(
    model = model,
    processing_class = tokenizer,
    train_dataset = dataset,
    dataset_text_field = "text",
    max_seq_length = max_seq_length,
    dataset_num_proc = 2,
    packing = False, # Can make training 5x faster for short sequences.
    args = training_args,
)

trainer.train()

model.save_pretrained("qwen_teacher_finetune") # Local saving
tokenizer.save_pretrained("qwen_teacher_finetune")

```

6.4 构建蒸馏训练器

```

class KDTrainer(SFTTrainer):

    def __init__(self, *args, teacher_model=None, if_use_entropy=None,
**kwargs):

```



```

super().__init__(*args, **kwargs)
self.teacher_model = teacher_model
self.if_use_entropy = if_use_entropy

def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
    outputs_student = model(**inputs)

    with torch.no_grad():
        teacher_outputs = self.teacher_model(**inputs)

    loss = outputs_student.loss
    logits = outputs_student.logits

    with torch.no_grad():
        teacher_logits = teacher_outputs.logits

    # 如果教师模型和学生模型输出形状不匹配, 对学生模型进行padding或对教师模型进行截断
    # print(logits.shape, teacher_logits.shape)
    # print(type(logits), type(teacher_logits))
    # if logits is None or teacher_logits is None:

    kl = 0
    if isinstance(logits, torch.Tensor) and isinstance(teacher_logits,
torch.Tensor):
        if logits.shape[-1] != teacher_logits.shape[-1]:
            # gap = teacher_logits.shape[-1] - logits.shape[-1]
            # if gap > 0:
            #     pad_logits = torch.zeros((logits.shape[0],
logits.shape[1], gap)).to(logits.device)
            #     logits = torch.cat([logits, pad_logits], dim=-1)

            teacher_logits = teacher_logits[:, :, :logits.shape[-1]]

            labels = inputs['labels']
            # kl = compute_fkl(logits, teacher_logits, labels,
padding_id=-100, temp=2.0)
            kl = compute_rkl(logits, teacher_logits, labels,
padding_id=-100, temp=2.0)

    if self.if_use_entropy:
        loss_total = 0.5 * kl + 0.5 * loss
    else:
        loss_total = kl

```

```
        return (loss_total, outputs_student) if return_outputs else
loss_total
```

6.5 读取刚才训练好的教师模型

```
teacher, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "qwen_teacher_finetune",
    max_seq_length = max_seq_length,
    dtype = dtype,
    load_in_4bit = load_in_4bit,
)

FastLanguageModel.for_inference(teacher)
```

6.6 加载学生模型

记得挂LoRA

```
# 学生模型
student, _ = FastLanguageModel.from_pretrained(
    # Can select any from the below:
    # "unsloth/Qwen2.5-0.5B", "unsloth/Qwen2.5-1.5B", "unsloth/Qwen2.5-3B"
    # "unsloth/Qwen2.5-14B", "unsloth/Qwen2.5-32B", "unsloth/Qwen2.5-72B",
    # And also all Instruct versions and Math. Coding versions!
    model_name = "unsloth/Qwen2.5-1.5B",
    max_seq_length = max_seq_length,
    dtype = dtype,
    load_in_4bit = load_in_4bit,
    # token = "hf_...", # use one if using gated models like meta-
llama/Llama-2-7b-hf
)

student = FastLanguageModel.get_peft_model(
    student,
    r = 16, # Choose any number > 0 ! Suggested 8, 16, 32, 64, 128
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj",],
    lora_alpha = 16,
    lora_dropout = 0, # Supports any, but = 0 is optimized
```

```

    bias = "none",      # Supports any, but = "none" is optimized
    # [NEW] "unsloth" uses 30% less VRAM, fits 2x larger batch sizes!
    use_gradient_checkpointing = "unsloth", # True or "unsloth" for very long
context
    random_state = 3407,
    use_rslora = False, # We support rank stabilized LoRA
    loftq_config = None, # And LoftQ
)

student.print_trainable_parameters()

```

6.7 蒸馏训练

```

# 训练参数
args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=10,
    do_train=True,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=16,
    logging_steps=10,

    save_strategy='epoch',
    save_total_limit=10,
    bf16=True,
    learning_rate=0.0005,
    lr_scheduler_type='constant',
    optim="adamw_torch_fused",
)

trainer = KDTrainer(
    model=student,
    teacher_model=teacher,

    if_use_entropy=True,
    processing_class=tokenizer,
    train_dataset=dataset,
    dataset_text_field = "text",
    max_seq_length=max_seq_length,
    dataset_num_proc=2,
    packing=False, # Can make training 5x faster for short sequences.
)

```

```
args=args,  
)  
  
# 如果是初次训练resume_from_checkpoint为false, 接着checkpoint继续训练, 为True  
trainer.train(resume_from_checkpoint=False)
```